

Real-Time Classification of Fraudulent Reviews Using Supervised and Deep Learning

Göksel OKANDAN

Computer Engineering

Yıldız Technical University

Istanbul, Türkiye

goksel.okandan@std.yildiz.edu.tr

Abstract—User reviews significantly influence consumer decision-making on online commerce platforms. Consequently, malicious actors ranging from business establishments to ideological groups frequently deploy computer-generated fraudulent reviews to artificially damage competitors' reputations or suppress legitimate public backlash. In order to protect consumers and preserve the digital integrity of online platforms, this study presents the development of a machine learning-based detection system designed to accurately distinguish between authentic human opinions and synthetic, computer-generated reviews. The methodology evaluates four models based on three distinct algorithms: Naive-Bayes, k-Nearest Neighbor (k-NN), and a Bidirectional Long-Short Term Memory (LSTM) neural network. These models were trained on a balanced dataset of 40,432 pre-labeled reviews and deployed within a custom desktop application for real-time classification and performance tracking. The experiment results indicate that the LSTM-based model achieved the highest predictive performance with an F1-score of 91.78%. However, the baseline Naive-Bayes algorithm also proved highly efficient, delivering a competitive F1-score of 86.54%. The study highlights the important trade-off between high detection accuracy and computational computing costs in fraud/spam detection systems.

Index Terms—fraud detection, supervised learning, deep learning, fraud reviews, algorithm comparison

I. INTRODUCTION

Human decision-making has always been affected others' opinions when it came to any matter that involved multiple people, either in business or personal matters. This could be seen best in this age of information and communication we are currently living in, where one person from any place on Earth could influence someone else's opinion. One of the biggest displays of this phenomena in today's world can be seen when we're buying a product or a service, where we put importance on the quality, price, hospitality and other matters, in the form of user reviews. What people think of their products/services is also important for the business establishments as well. Therefore, in today's worldwide economy, user reviews play a big role for all establishments from big, multi-national titans to small, single person or family owned ones [1].

As a result of this environment where user reviews can affect people's decision-making about products/services, malicious actors like business rivals of an establishment or ideologically-driven groups can engage in the process of creating a surge of multiple negative reviews about a business' practices despite

not being truthful. This process is generally called "bot-driven fraud" and, similar to another practice called "review bombing" where it's done by human users, it's proven itself to be genuinely damaging to a business establishment's revenue and financial stability [2].

The motivations for performing these kinds of fraudulent activities are considered to be rather multifaceted. For example, a business establishment that's facing public backlash about their products may engage in this kind of fraud in order to suppress or cover the backlash that they're facing. In contrast, malicious actors can engage in this type of bot-driven fraud in order to negatively affect the reputation of a competitor's reputation [3]. In both scenarios, the potential customer for a business establishment ends up being misled by deceptive actors distorting the market dynamics of today's economy and ends up punishing honest business establishments.

In order to protect consumers from misleading reviews and to preserve digital integrity of review platforms from these kinds of fraudulent activities, this project has aimed to create a machine learning system that is capable of accurately distinguishing between original reviews written by genuine humans and computer-generated, synthetic, deceptive reviews. In order to separate these reviews, the model makes use of four different models based on three different machine learning/deep learning algorithms for real-time review classification, those being Naive-Bayes, k-Nearest Neighbor and Long-Short Term Memory, with the models being deployed on a desktop application for a user's ease of use.

From this point forward, the subsequent sections of this report will detail the data acquisition and pre-processing methodology, the architectural design of the machine learning models, the integration of the desktop inference engine, and a comprehensive analysis of the final performance metrics.

II. METHODOLOGY

A. Preparation of the Project Environment

In order to create the project and evaluation environment (that could provide the same conditions for all models, a crucial fact for fair results), a two-stage environment that consisted of a Google Colab Python 3 Runtime environment with an Nvidia Tesla T4 GPU, which featured 16 GB of VRAM, 2560 CUDA cores and 320 Tensor cores, and the developer's own local virtual environment which featured an

AMD Ryzen 7 7535HS processor with 32 GB of system RAM, along with an Nvidia RTX 4060 Mobile which has 8 GB of VRAM, 3072 CUDA cores and 96 Tensor cores. However, the GPU was not able to be utilized due to the local virtual environment not having a GPU-compatible version of the Tensorflow library on Windows 11.

Despite all that however, this two-stage project and evaluation environment has proven itself to be crucial, especially as the models trained in the Colab environment, having used the legacy versions of the Keras and scikit-learn libraries, being incompatible with the newer versions of the same libraries used in the desktop application.

In addition to all the libraries mentioned during the preparation of the project and evaluation environment, this project also used the following libraries: **os** to load and read files from the filesystem, along with saving and writing files to the same filesystem, **customtkinter** to create the desktop GUI application due to its capabilities (such as dark mode) and ease of use compared to the base **tkinter** library, **PIL** for saving images to variables as artifacts, along with embedding the plots generated into the "Results" and "Archived Results" screens, **Pandas** to read data from dataset's .csv files and use them both during the training session, as well as in the desktop application, **Numpy** to perform the necessary mathematical operations in the dataset, along with finding assigning the true and false positives in the relevant functions, **matplotlib** and **seaborn** to be able to create, manage and export plots generated in the training phase and the desktop application, **threading** and **queue** to implement multi-threading in the desktop application, **time** and **datetime** to save timestamps and perform necessary date-related actions, **psutil** to get memory readings in the "performance mode" tests, **joblib** and **pickle** to save and load the trained models and helpers as artifacts in the desktop application.

With all the necessary implementations of these libraries, the project and evaluation environment was ready.

B. Choosing the Dataset and the Pre-processing Procedure

In order to find a suitable dataset, it had to fulfill several conditions, such as being labeled as "original" or "computer generated", allowing binary classifications, having the necessary metadata in order to allow easier navigating of the examples in the "Dataset Examples" screen etc.. One such dataset was the **Fake Reviews Dataset** from the user "MuqadasEjaz", that included 40432 pre-labeled reviews with a nearly 50/50 split that allowed for binary classification, along with metadata such as the category and rating that allowed further analysis. After further review of the dataset, it was decided that this dataset would be the dataset used in this project.

After downloading the dataset from Kaggle using the **Kagglehub** library, the dataset was pre-processed using the **nlTK** library, which included the processes of lowercase conversion, punctuation and number removals in order to unify words ("Library", "library." and "library" being the same for example), and the removal of stopwords included in the **nlTK**

library's stopwords list to remove insignificant words that would contaminate the training process (such examples being words like "is", "are", "for", and "any").

After the pre-processing and "cleaning" was completed, the dataset was then split 80/20 into train and test sets. Following this, the dataset was vectorized using a TF-IDF vectorizer with a "max_features" variable of 10000 in order to provide the vectorizer for training the Naive-Bayes and k-NN based models, along with being tokenized with a Tokenizer, with the *max_words* variable having a value of 10000, for training the LSTM based model. The vectorizer and the tokenizer were exported afterwards to be used to train the models afterwards.

After all the necessary actions related to the dataset was completed, the processed dataset was exported as "*cleaned_fake_reviews.csv*", ready for training the models.

C. The Base Algorithms and Their Implementation Process

As stated in the project document, implementing three base algorithms were necessary for the success of this project. After researching which models would suit the purposes of this project, the algorithms of Naive-Bayes as the baseline algorithm, and k-Nearest Neighbor as the alternative were decided upon, along with the Long-Short Term Memory (LSTM) deep learning algorithm as the advanced algorithm.

1) *Baseline Algorithm: Naive-Bayes*: The Naive-Bayes algorithm is a type of probabilistic machine learning classifier that is based on the Baye's theorem. It's generally considered to perform great in several real-world applications, such as spam detection, document categorization and sentiment analysis [4] [5]. It assumes all features inside the dataset are independent from each other and they are all equally important, hence the "Naive" part of this algorithm's naming. Additional assumptions include the normal distributions of continuous features and multinomial distributions of discrete features. It also assumes that there is no missing data in the dataset.

The Naive-Bayes algorithm also has a test runtime and general space complexity of $O(dc)$, where 'd' is number of features and 'c' is the number of classes, with the same algorithm having a training runtime complexity of $O(Nd)$, where 'N' is the total number of data points and 'd' is the number of features [6] [7].

2) *Alternative Algorithm: k-Nearest Neighbor*: k-Nearest Neighbor is a machine learning algorithm that is generally used for classification and regression tasks. It works by identifying the k numbers of closest data points based on a distance metric, Euclidean, Manhattan, or Minkowski in general, and making predictions based on these data (hence, the 'k' at the start of its name) [9] [8]. It assumes that the closer the data points are, the more they are relevant to each other without exception. It also assumes that the distance metric provided to it is the best distance metric for its purpose and that all data are represented as points in a metric space, so it can calculate the distance between these representation points [10]. However, it does not make assumptions, unlike Naive-Bayes and several other machine learning algorithms, about the distribution of

data points provided to it [11]. This algorithm is also prone to suffering from a "Curse of Dimensionality" where a high number of features/dimensions leads to data sparsity, which reduces the performance of the algorithm.

This algorithm has a runtime complexity of $O(1)$ during training, and $O(Nd)$ during brute-force testing, where 'N' is the number of data points and 'd' is the number of features/dimensions. Since it has to load all the data points, it has a space complexity of $O(Nd)$ during both training and testing.

3) *Advanced Algorithm: Long-Short Term Memory (LSTM)*: The Long-Short Term Memory (LSTM) is a sub-type of a Recurrent Neural Network (RNN), where it's specifically designed to learn from sequential data, an area where traditional RNN's tend to struggle. They are especially effective for tasks involving patterns across time due to their ability to remember information for a longer amount of time with their memory-cell structure. Because of this, they are generally used for tasks involving sequential data, which include speech recognition and analysis, handwriting analysis and future guesses, such as the stock brokerage and weather forecasting [12]. The most basic LSTM structure consists of an input, output and forget gate, an input block, an output activation function, a peephole connection and a memory block called cell [13].

LSTM's only tend to assume that the historical/past inputs provided to them affect current or future events, therefore they have a soft-necessity of being provided sequential data. They also have a general time complexity of $O(Wh^2)$ per time step where 'W' represents the weight transformations, and 'h' is the number of hidden units, ending with a total time complexity of $O(Th^2)$, where 'T' is the length of the time sequence. It has total space complexity of $O(h^2)$, to store network parameters and hidden units [14] [15].

4) *The Training and Implementations of Algorithms*: In order to train models based on these algorithms, a Jupyter Notebook was used to isolate all environments from the effects of other algorithms. Additionally, in order to prove that the "random_state" modifier used didn't just allow a stroke of luck, the resampling technique of **K-Fold Cross Validation** was used, with variable "K" being 5 in this instance.

To train the Naive-Bayes based model, the cleaned dataset was loaded, re-split into the same train/test sets via the use of "random_state" variable, and vectorized by the TF-IDF vectorizer created during the pre-processing session. Afterwards, a *MultinomialNB* classifier was created with the *alpha* (Laplace Smoothing variable) value of 1, and the model was trained and tested before active use in the desktop application. The results can be found in the III (**Results**) part of the report.

After training the Naive-Bayes based model, two different k-NN based models were trained, one with 10000 features as originally planned and one with 100 features (reduced by TruncatedSVD) in order to view the effects of Dimensionality Reduction. Additionally, in order to optimize the training values, neighbor counts from 1 to 21 were tested to get the highest accuracy rating. The optimum values were then determined to be 9 for 100 feature model, and 2 for the 10000 feature

model. Afterwards, the same training process as the Naive-Bayes based model (excluding the k-NN specific processes like TruncatedSVD feature reduction) was performed again, and the resulting two models were saved to be in the desktop application. The results, as it was with the Naive-Bayes, can be found in the III (**Results**) part of the report.

After training the Naive-Bayes and k-NN based models, the LSTM based model followed suit. After all the processes with the dataset was performed (loading and splitting the dataset with the *random_state* variable, tokenizing and padding the split dataset), a Sequential Bidirectional LSTM was defined with the variables of *input_dim = 10000*, *output_dim = 64*, *dropout = 0.2*, along with a Sigmoid activation function and a custom optimizer with a learning rate of 0.001. In addition to all of these, an early stopping protocol was set with a patience value of 3 epochs. After several tests, the optimum amount of epochs was then found out to be 3 (the validation loss increased exponentially after 3 epochs) with a batch size of 32. The training was then commenced with the above variables set, and the process resulted in the creation of the LSTM model that would be used in the desktop application.

D. Desktop GUI and the Basic Use Case Scenario

1) *Dataset Manager*: The "Dataset Manager" screen (shown as "1" in the Fig.1) is used in order to explore the dataset's statistics and examples, navigated by the two buttons above the frames. The "Statistics" screen (navigated to the by the button shown as "2" in the Fig.1), shows the amount of total reviews loaded, the split of these reviews between "Original" and "Computer Generated" by count and percentage, and the shape of the dataset. In addition to these statistics, the "Examples" screen (navigated to the by the button shown as "2" in the Fig.1) allows the exploration of the dataset reviews (with a maximum of 100 reviews each filter) via filters above the frame (as they are seen in Fig.2).

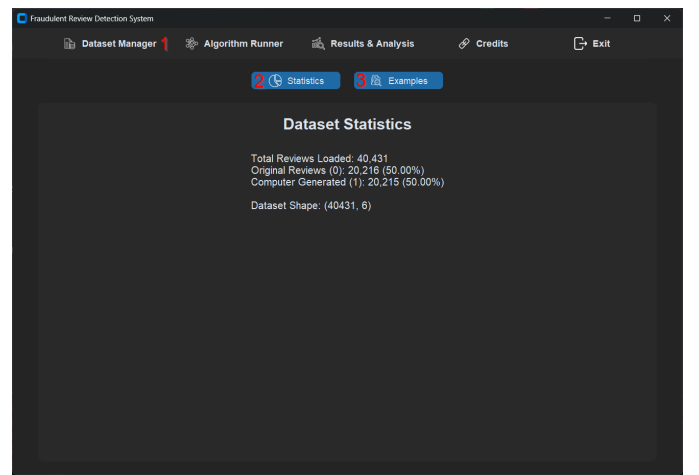


Fig. 1. The "Dataset Manager" screen. In this screen, the user is able to view the statistics of the dataset, along with filtered examples before their pre-processing.

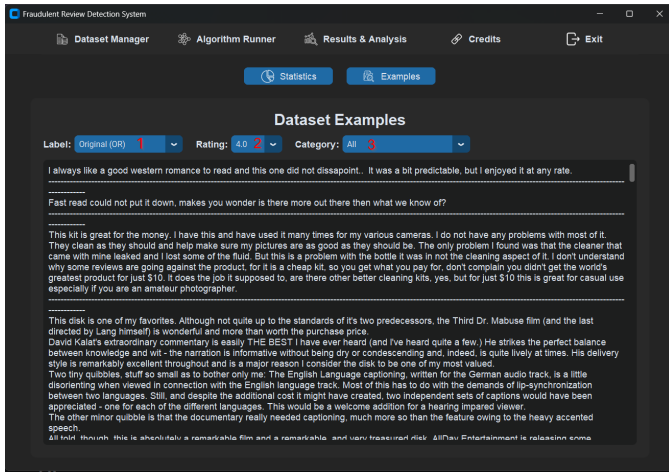


Fig. 2. The "Dataset Examples" screen. In this screen, the user is able to explore the examples of a specific filtering up to 100 reviews.

2) *Algorithm Runner*: The "Algorithm Runner" screen is used to perform tests with the afore-mentioned dataset and trained models. There are two ways to perform tests in this application, "Base Mode" and "Performance Mode" (can be seen in Fig.3 as the square marked with "1"). "Base Mode" tests are done in the way that was mentioned in the project document: the user selects the models that they want to use in the test (the area marked by "2" in Fig.3), presses the "Start Base Test" button (the button with "3" in Fig.3), resulting in the creation of the area marked with "4" in Fig.3, which includes the progress bar indicating the progress of the testing process, along with the accuracy metrics of the model with the addition of runtime.

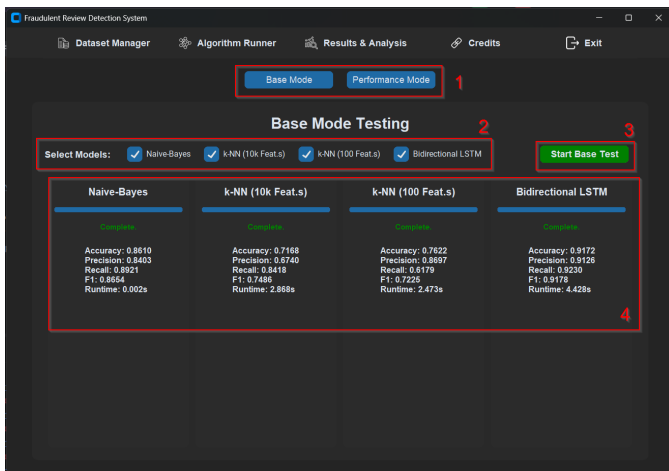


Fig. 3. The "Algorithm Runner" screen. Here, the users can run "base mode" with their selection of models, or a "performance mode" test that compares every model extensively.

After the test is concluded, the "Results" page (Fig.4) is shown to the user, which includes a performance comparison chart, along with all the models ranked according to their F1-Scores (the area marked with "1"). In a model's specific page,

the user able to see the model's overall rank along with their metrics, flagging examples and the confusion matrix of the relevant model (areas "2", "3" and "4" in Fig.4, in order of mention).

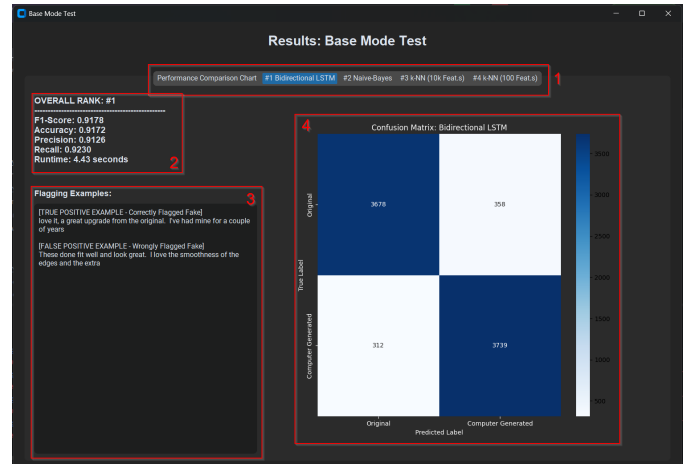


Fig. 4. The "Results" screen presented at the end of a test. In this screen, the results are presented with the models ranked according to their F1-Scores.

In an alternative to "Base Mode" tests, the user is also capable of performing "Performance Mode" tests. These tests are similar to "Base Mode" tests in general, with the exceptions of being done on 25%, 50% and 100% test data splits, having no configurations, and recording the memory use and showing it. After the conclusion of these tests, the results are shown in a similar way to the "Results" screen of the "Base Mode" tests.

3) *Results & Analysis*: This screen is used to view past results of performed tests, in order to be able to understand the current test results better. The user is able to press the "View Report" button in order to access the results of prior tests (pictured in Fig.5), where they are greeted with the "Archived Result" screen. This screen has the logs of the prior test on the left side (marked with "1" on Fig.6), along with the plots generated during that test on the right side (marked with "2" on Fig.6).

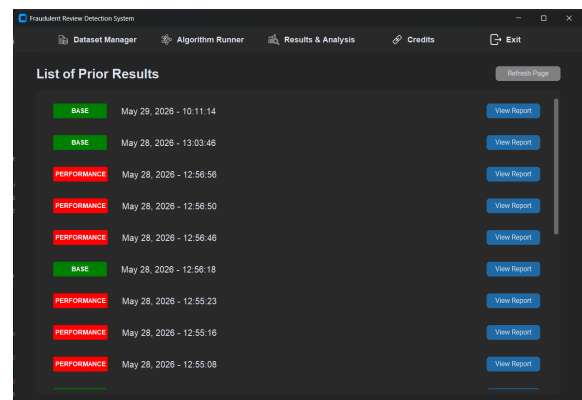


Fig. 5. The "Results & Analysis" screen, where users can choose a prior test result in order to better understand the current tests.

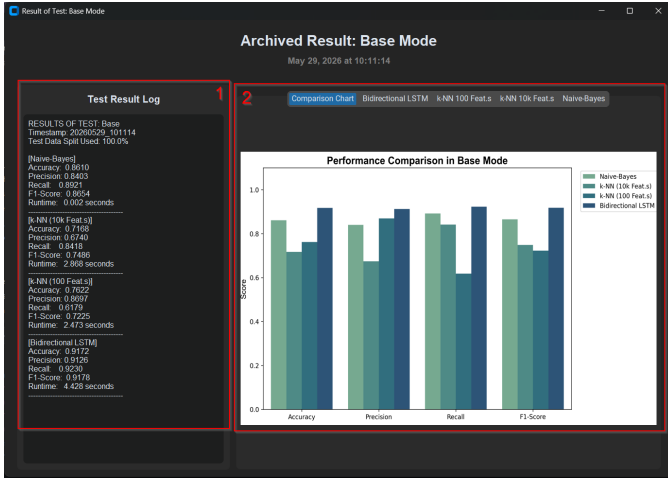


Fig. 6. The "Archived Result" screen, where all the results of a prior test are shown.

4) *The Basic Use Case for a User:* The basic use case for a user would be the following process:

- 1) The user opens the app and sees the "Credits" screen.
- 2) The user moves to the "Dataset Manager" screen, and views the statistics and examples of the dataset.
- 3) The user moves to the "Algorithm Runner" screen, selects the "Base Mode" option.
- 4) The user selects the models that they want to use for the test out of the four options and presses the "Start Base Test" button.
- 5) The user monitors the ongoing test (it generally takes 15-20 seconds at most) and waits for it to end.
- 6) When the test is finished, the user is able to view the results of the current test
- 7) (Optional) The user compares the test with past tests, using the "Results & Analysis" screen.

Additionally, the system architecture diagram for this project, which includes the data flow from input to output can be found below, in Fig.7.

III. RESULTS

TABLE I
MODEL TRAINING TIME RESULTS

Algorithm Name	Training Time (sec.s)	Testing Time (sec.s)	Metrics
Naive-Bayes	0.007	0.002	Fig.8
k-NN (10k)	0.002	2.922	Fig.8
k-NN (100)	0.003	0.432	Fig.8
LSTM	133.649	4.700	Fig.8

As it can be seen from the comparison chart (Fig.8), the confusion matrices (Fig.9) and the table (Table I), the Bidirectional LSTM-based model has performed the best among all four models, with the F1-score of 0.9178 (91.78%). It is followed by the "chosen as baseline" algorithm of Naive-Bayes with the F1-score of 0.8654 (86.54%) while being extremely fast compared to the Bidirectional LSTM-based model with an overall training and testing time of less than 0.01 seconds

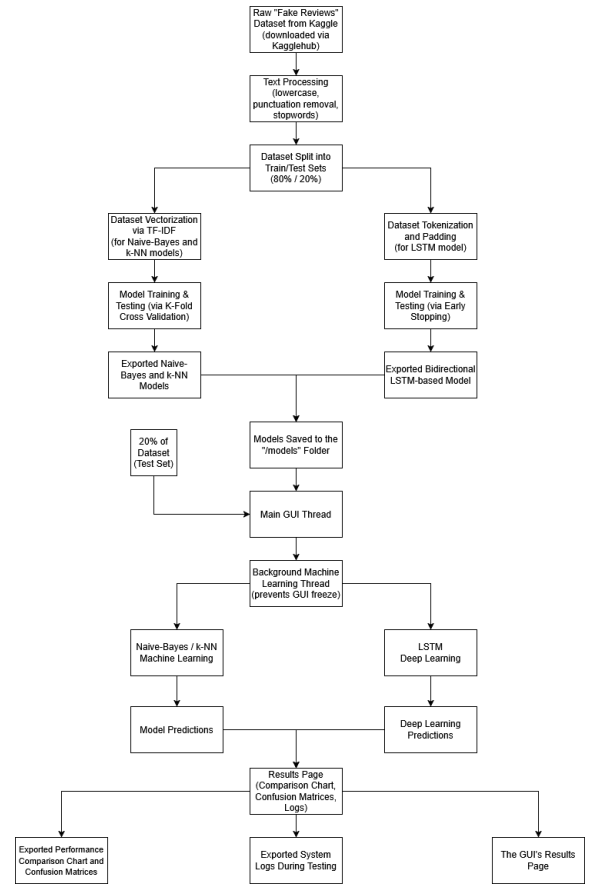


Fig. 7. The system architecture chart for the project, starting from data acquisition to ending with the desktop application's test results.

(0.007 for training and 0.002 for testing), compared to the LSTM-based model's multiple minutes of training and testing.

However, when it comes to the k-NN based models, there is a wide performance gap between these models and the Naive-Bayes and LSTM-based models, with k-NN based models having F1-scores of 0.7486 (74.86%) (10000 features) and 0.7225 (72.25%) (100 features). However, these models have performed nearly the opposite of each other, despite being based on the same algorithm. The 10000 feature model has performed with a precision score of 0.6740 (67.40%) and a recall score of 0.8418 (84.18%), while the 100 feature model has performed with a precision score of 0.8697 (86.97%) and a recall score of 0.6179 (61.79%). Accuracy-wise, the 100 feature model has performed better, with an accuracy score of 0.7622 (76.22%) compared to the 10000 feature model's 0.7168 (71.68%).

When it came to testing on the desktop application however, no performance change was detected, probably to due being tested on the same testing dataset since the **random_state** variable was the same value. Nevertheless the mean cross-validation accuracy values were similar compared to their actual accuracy scores, mostly having a difference up to 0.01 (1%), therefore proving the fact that the **random_state** variable used just didn't cause a stroke of luck.

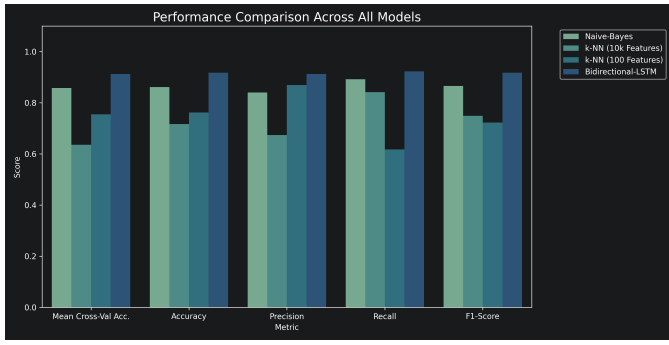


Fig. 8. The performance comparison chart with the traditional metrics of the models (accuracy, precision, recall, F1-score) along with mean cross-validation accuracy being compared against each other after training.

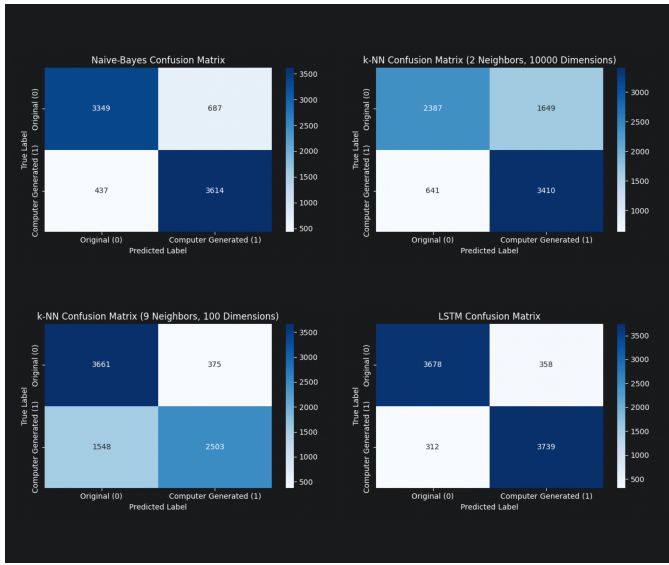


Fig. 9. The confusion matrices of all the models trained locally (Naive-Bayes on the top left, k-NN with 10000 features on the top right with the k-NN with 100 features on the bottom left, along with the Bidirectional LSTM's on the bottom right).

IV. DISCUSSION

The tests done with the models in this project has caused the rise of two questions/dilemmas in the field of fraud/spam detection: "Is it better to have near-perfect results at the cost of increased computation requirements" and "Should the detection system focus on better fraud detection at the cost of genuine messages?". In the specific field of this project, the conclusion for the first question is generally considered to be "yes", due to the advanced algorithm of LSTM's having marginally increased computational costs compared to the baseline algorithm of Naive-Bayes.

However, the second question's answer is generally thought to be "no", due to the potential of causing outrage in customers not being able to share their opinions due to impacts of being flagged for sharing fraudulent reviews (such as account restrictions or bans), therefore causing decreased use of the review system.

Nonetheless, the baseline algorithm of Naive-Bayes having a near 85% success rate proves the project has an acceptable rate of success, while still needing to be validated by more review data that is similar to the dataset used in this project.

V. CONCLUSION

This project presented the design and development process of a fraudulent review detection system that intends to solve the problem misguiding the public over a product's quality that is caused by multiple reasons. In order to solve this problem, a fraudulent review system that employs multiple machine learning models has been proposed. In the future of this project, multiple improvements are planned, which include but are not limited to: the addition of more models, including CatBoost and XGBoost, an expanded results presentation with more metrics being recorded (including memory usage), and the rewriting of the desktop application in PyQt6, therefore complimenting the expanded options by improving the user's experience with added QoL features. With these options, it is planned for this project to contribute further into the fraud review and cybersecurity literature and products.

ACKNOWLEDGEMENT

LLM assistance (Gemini 3.1 Pro) was used during the development process of this project, primarily for guidance and coding during the desktop application coding process.

Additionally, I would like to thank the Flaticon website [16] and the icon creators Artifex, Becris, muh zakaria, Shashank Zingh, Freepik, gravisio, Pixel perfect and iconixar for creating and hosting the icons used in this project.

REFERENCES

- [1] Phamthi, V. A., Nagy, Á., & Ngo, T. M. (2024). The influence of perceived risk on purchase intention in e-commerce—Systematic review and research agenda. *International Journal of Consumer Studies*, 48(4), e13067. <https://doi.org/10.1111/ijcs.13067>
- [2] Xia, R., Dong, X., An, J. et al. The impact of fake online reviews on customer satisfaction: an empirical study on JD.com. *Electron Commer Res* 25, 4689–4716 (2025). <https://doi.org/10.1007/s10660-024-09865-y>
- [3] Tetik, M. When bots mislead markets: asymmetric contamination risk in sentiment-based investment decisions. *Futur Bus J* 12, 156 (2026). <https://doi.org/10.1186/s43093-026-00863-0>
- [4] <https://www.sciencedirect.com/topics/mathematics/naive-bayes>
- [5] <https://www.geeksforgeeks.org/machine-learning/naive-bayes-classifiers/>
- [6] <https://towardsdatascience.com/understanding-naive-bayes-algorithm-f9816f6f74c0/>
- [7] <https://stats.stackexchange.com/questions/96995/machine-learning-classifiers-big-o-or-complexity>
- [8] <https://www.geeksforgeeks.org/machine-learning/k-nearest-neighbours/>
- [9] <https://towardsdatascience.com/the-math-behind-knn-3d34050efb71/>
- [10] https://www.cs.cornell.edu/courses/cs4780/2018sp/lectures/lecturenote02_kNN.html
- [11] <https://www.sciencedirect.com/topics/biochemistry-genetics-and-molecular-biology/k-nearest-neighbor>
- [12] <https://www.geeksforgeeks.org/machine-learning/lstm-networks/>
- [13] <https://www.sciencedirect.com/topics/engineering/long-short-term-memory>
- [14] <https://chinougea.medium.com/lstm-computational-complexity-d5d8f72e6db4>
- [15] <https://ai.stackexchange.com/questions/33927/what-is-the-time-complexity-for-testing-a-stacked-lstm-model>
- [16] <https://www.flaticon.com/>